Machine Learning Project

Kevin Sony

Summer 2025

1 Introduction

We want to create a simple machine learning library in C. This is to both relearn how C works, and to fully understand the mathematics behind machine learning.

There are some things we want from this library. It should be a dynamic library so that wrappers can be written in Python, Java, and other languages. There should be an intuitive way to construct these wrappers, so that it does not get too complicated to implement. Finally, the library itself should be intuitive to read and build, so that others can potentially debug if necessary.

Some things to hopefully implement:

- Linear/Logistic Regression
- K-Nearest Neighbors
- K-Means Clustering
- Artificial Neural Networks

The basic premise of machine learning algorithms is for some given inputs, we want to find out some information from this output, whether it be a classification or a prediction. Specifically, machine learning algorithms construct a function f that should take in an input x from a specific input space, and return the y we expect to see.

For example, if we wanted to classify images as animals, humans, cars, and other objects, the function's domain is the space of images of all the objects that are in the classifier, and the function's range is a probability vector, containing the probabilities it is any one of the specified objects. So if we pass in an image of a dog, we want our neural network to return a vector indicating the object in the image has a high probability of being a dog, and a low probability of being anything else in the classifier.

To be able to construct such a function, we first need to find some data to construct it from. From there, we check its validity by testing on other data not used to construct it.

2 Artificial Neural Networks

ANNs construct the function f through applying affine then nonlinear transformations multiple times.

We start with an input $x_1 \in \mathbb{R}^{m_1}$, apply an affine and nonlinear transform to turn it to an output $y_1 \in \mathbb{R}^{m_2}$. This then becomes the new input $x_2 = y_1$. By applying this transformation $n-1$ more times, we get the output $y_n \in \mathbb{R}^{m_{n+1}}$. In other words,

$$\begin{aligned} y_1 &= a_1(W_1 x_1 + b_1) \\ y_2 &= a_2(W_2 x_2 + b_2) \\ &\vdots \\ y_n &= a_{n-1}(W_{n-1} x_{n-1} + b_{n-1}) \end{aligned}$$

where $W_1, \cdots, W_{n-1}$ are weights, $b_1, \cdots, b_{n-1}$ are biases, and $a_1, \cdots, a_{n-1}$ are nonlinear "activation" functions. This defines a function $\hat{f}$, where $\hat{f}(x_1) = y_n$.

It doesn't do much good to simply have a function $\hat{f}$. We want y_n to equal a true value t, i.e. we want to find a function f such that $f(x_1) = t$. If $\hat{f} = f$, we are done, but if not, we need to modify $\hat{f}$ to get closer to f, and this is done by modifying its parameters, the weights and biases.

There are several ways to solve for the optimal weights, and all of them involve trying to minimize a loss function L that takes in y_n and t and outputs how far off the two vectors are. From there, one can either try minimizing the loss function through various methods such as gradient descent or Newton's method. They each have various trade-offs. Newton's method converges quadratically, but it requires convexity (at the very least, local convexity if not global), and the Hessian can be quite computationally expensive to compute. Gradient descent converges linearly, but it is much cheaper to compute gradients.

This library will use the Mean Square Error as the loss function, and gradient descent for solving for the optimal weights and biases that minimize it. The MSE loss function is

$$\begin{aligned} L(y_n, t) &= \sum_{i=1}^{i=m_{n+1}} (\hat{f}(x_1)[i] - f(x_1)[i])^2 \\ &= \sum_{i=1}^{i=m_{n+1}} (y_n[i] - t[i])^2 \end{aligned}$$

The derivative of L with respect to the given output is

$$\frac{\partial L}{\partial y_n} = \nabla_{y_n} L$$

Let $z_{n-1} = W_{n-1} x_{n-1} + b_{n-1}$. Then $y_n = a_{n-1}(z_{n-1})$ and

$$\frac{\partial y_n}{\partial z_{n-1}} = a'_{n-1}(z_{n-1})$$